

PHASE 10 — ENTERPRISE SCANNING ENGINE & MULTI-SOURCE DISCOVERY FABRIC

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 10 of 22

Document Type: Technical Research & Engineering Handbook

Status: Enterprise-Scale Discovery & Scanner Architecture

Table of Contents

1. Purpose of This Phase
2. What ECDAT Must Actually Scan
3. Discovery Is a Multi-Layer Problem
4. The Discovery Fabric
5. Scanner Architecture
6. Scanner Types
7. Source-Code Scanner
8. Repository Scanner
9. Binary Scanner
10. Library Scanner
11. Container Scanner
12. Package Scanner
13. Configuration Scanner
14. Certificate Scanner
15. Network/TLS Scanner
16. Cloud Scanner
17. Kubernetes Scanner
18. HSM/KMS Scanner
19. Runtime Scanner
20. SBOM Integration
21. CI/CD Integration

22. Endpoint Agent
23. Passive Discovery
24. Active Discovery
25. Offline Discovery
26. Air-Gapped Environments
27. Scan Orchestration
28. Job Queue
29. Distributed Workers
30. Scan Prioritization
31. Incremental Scanning
32. Scheduled Scanning
33. Event-Driven Scanning
34. Scan Lifecycle
35. Repository Scan Lifecycle
36. Binary Scan Lifecycle
37. Container Scan Lifecycle
38. Network Scan Lifecycle
39. Evidence Collection
40. Evidence Normalization
41. Evidence Deduplication
42. Asset Identity
43. Asset Fingerprinting
44. Version Tracking
45. Change Detection
46. Scanner Confidence
47. Coverage Metrics
48. Scanner Health
49. Failure Recovery
50. Retry Strategy
51. Credential Management
52. Secrets
53. Connector Security
54. Repository Authentication

- 55. Cloud Authentication
 - 56. Network Permissions
 - 57. Least Privilege
 - 58. Data Protection
 - 59. Air-Gapped Architecture
 - 60. Large Enterprise Scale
 - 61. Performance
 - 62. Parallelism
 - 63. Caching
 - 64. Storage
 - 65. Message Bus
 - 66. API Architecture
 - 67. Scan Result Pipeline
 - 68. Scanner-to-CBOM Flow
 - 69. Scanner-to-Graph Flow
 - 70. Scanner-to-AI Flow
 - 71. Scanner-to-Risk Flow
 - 72. Scan Prioritization Example
 - 73. Enterprise Example
 - 74. Sample Scan Result
 - 75. Coverage Dashboard
 - 76. Architecture
 - 77. Security Threat Model
 - 78. Requirements Derived From Phase 10
 - 79. Phase 10 Summary
 - 80. Phase 11 Preview
-

1. Purpose of This Phase

Phase 9 established the:

Cryptographic Knowledge Graph.

We now need to solve the practical question:

How does ECDAT discover all these assets in the first place?

The original problem statement explicitly expects ECDAT to scan:

- Source code repositories
- Binaries
- Libraries
- Container images

But a genuinely enterprise-grade implementation should go beyond those four.

ECDAT should ideally discover cryptographic artifacts across:

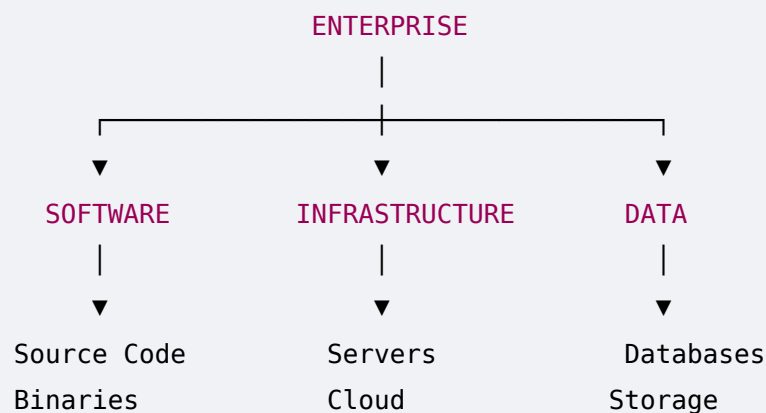
```
Source Code
Binaries
Libraries
Containers
Packages
Certificates
Configurations
Network Services
Cloud Infrastructure
Kubernetes
HSM/KMS
Runtime Systems
SBOMs
CI/CD Pipelines
```

Therefore Phase 10 designs the:

Enterprise Discovery Fabric.

2. What ECDAT Must Actually Scan

The discovery surface can be represented as:



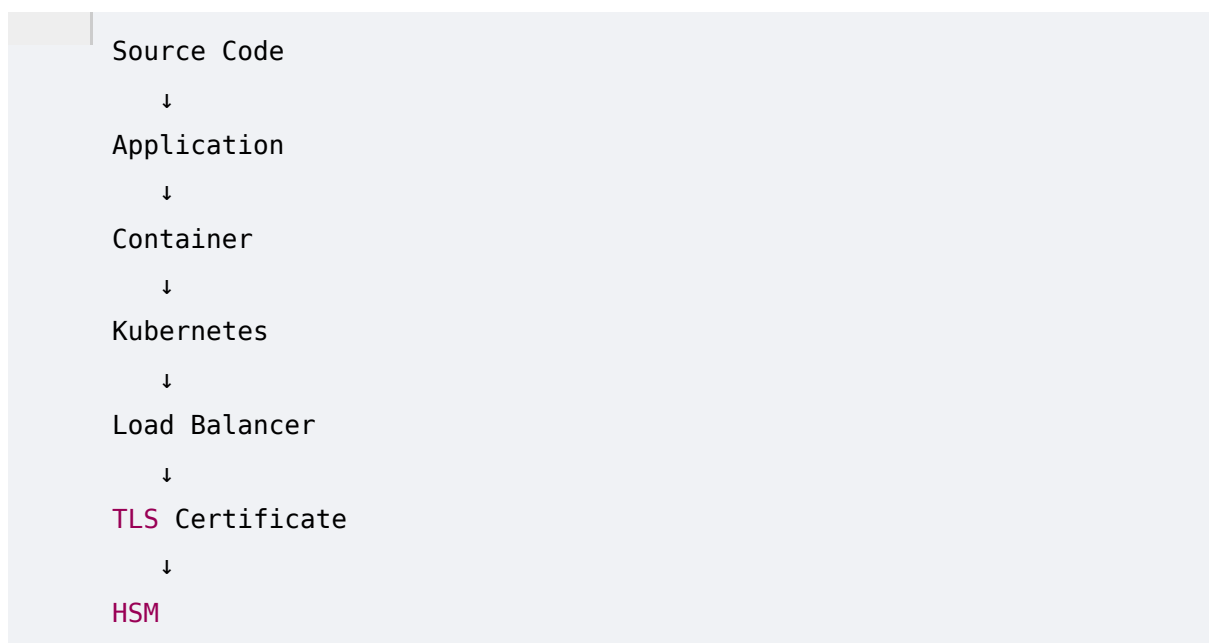
Libraries	Containers	Archives
Packages	Kubernetes	
Certificates	HSM/KMS	
Configurations	Network	

ECDAT must not assume cryptography exists only inside application source code.

3. Discovery Is a Multi-Layer Problem

A single scanner cannot reliably discover everything.

Consider:



Each layer exposes different evidence.

Therefore:

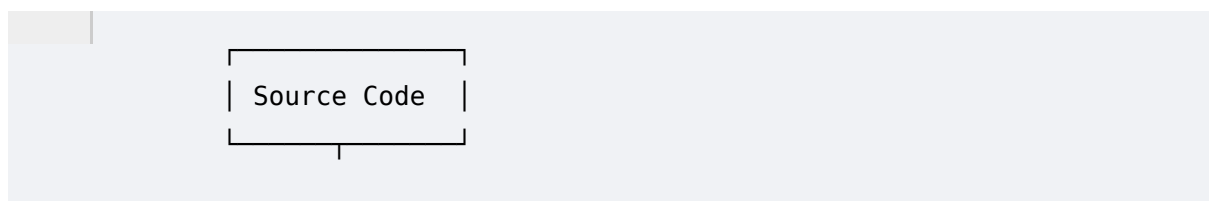


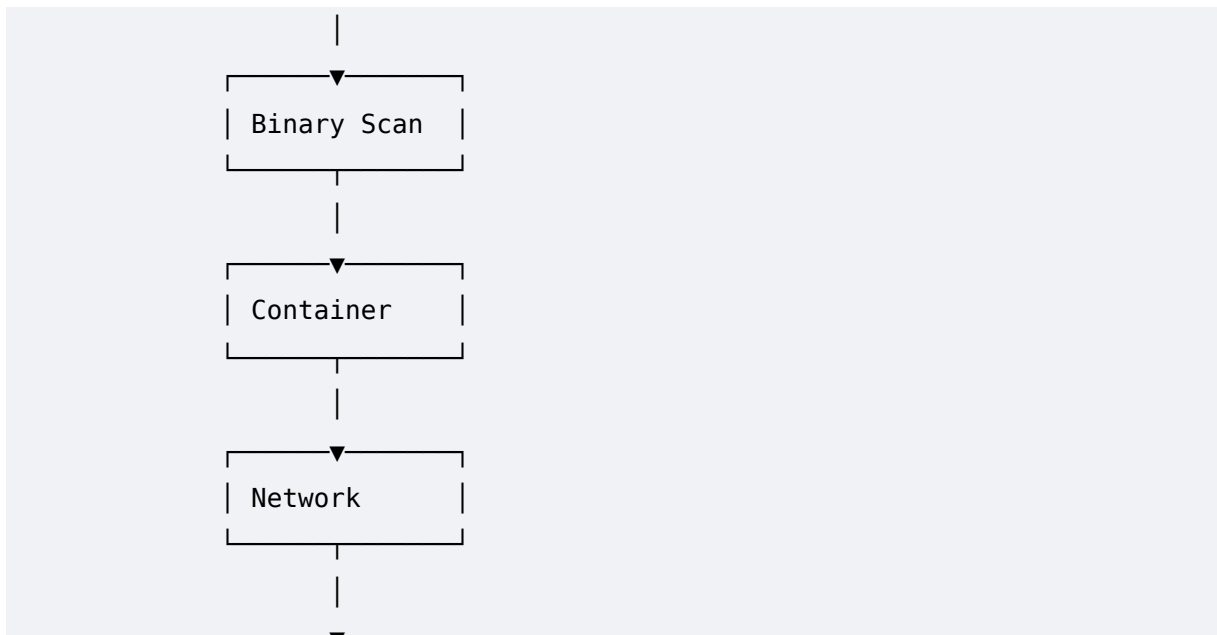
is fundamental.

4. The Discovery Fabric

The discovery fabric acts as the ingestion layer for ECDAT.

Conceptually:





Each scanner is specialized.

5. Scanner Architecture

A scanner should be treated as a modular plugin.

Generic interface:

```
Scanner
|
├─ discover()
├─ collect()
├─ analyze()
├─ normalize()
└─ emit()
```

Every scanner should produce standardized evidence.

6. Scanner Types

ECDAT can eventually support:

1. Repository Scanner
2. Source Scanner
3. Binary Scanner
4. Library Scanner
5. Container Scanner
6. Package Scanner
7. Configuration Scanner

8. Certificate Scanner
9. Network Scanner
10. Cloud Scanner
11. Kubernetes Scanner
12. HSM/KMS Scanner
13. Runtime Scanner
14. SBOM Scanner
15. CI/CD Scanner
16. Endpoint Scanner

Not all need to be built for the SIH prototype.

A strategic MVP should prioritize the highest-value scanners.

7. Source-Code Scanner

The source scanner analyzes:

```
.py  
.java  
.c  
.cpp  
.go  
.rs  
.js  
.ts  
.cs
```

It should detect:

- Cryptographic APIs
- Algorithms
- Key generation
- Certificates
- Hashes
- Signatures
- Encryption
- Key exchange
- Randomness
- Custom crypto

It feeds:

Static Analysis

+

AI Analysis

8. Repository Scanner

The repository scanner operates at project level.

It should collect:

Repository

Branch

Commit

Language

Dependencies

Build Files

Configuration

Documentation

Potential integrations:

GitHub

GitLab

Bitbucket

Self-hosted Git

For SIH, generic Git repository support may be sufficient initially.

9. Binary Scanner

Binary analysis is critical because source code may not be available.

Potential inputs:

ELF

PE

Mach-0

Shared Libraries

Executables

Firmware

The scanner can inspect:

- Imported symbols

- Linked libraries
 - Strings
 - Embedded certificates
 - Crypto constants
 - Known APIs
 - Function signatures
 - Binary metadata
-

10. Library Scanner

Libraries are particularly important because one library may affect thousands of applications.

ECDAT should detect:

```
OpenSSL
BoringSSL
LibreSSL
Bouncy Castle
libsodium
wolfSSL
mbedTLS
```

and application-specific crypto dependencies.

11. Container Scanner

Container images should be analyzed before deployment.

Example:

```
Container
|
├─ Base Image
├─ Packages
├─ Shared Libraries
├─ Application
└─ Configuration
```

ECDAT can identify cryptographic libraries embedded in the image.

12. Package Scanner

Package manifests:

```
package.json  
pom.xml  
requirements.txt  
Cargo.toml  
go.mod  
*.csproj
```

can reveal dependencies.

This is especially useful for identifying indirect crypto libraries.

13. Configuration Scanner

Cryptography often lives in configuration.

Examples:

```
TLS versions  
Cipher suites  
Key sizes  
Certificate paths  
Algorithm names  
KMS configuration  
HSM configuration
```

Formats:

```
YAML  
JSON  
XML  
TOML  
INI  
Properties  
Environment Variables
```

14. Certificate Scanner

Certificate discovery is one of the highest-value capabilities.

ECDAT should identify:

- Certificate
- Algorithm
- Key Size
- Issuer
- Validity
- Usage
- Endpoint
- Environment

Potential sources:

- Repository
- Filesystem
- Container
- Network Endpoint
- Cloud Certificate Service
- Load Balancer
- Kubernetes Secret

15. Network/TLS Scanner

ECDAT can inspect externally reachable services.

Conceptually:

- Endpoint
 - ↓
 - TLS Handshake
 - ↓
 - Certificate
 - ↓
 - Protocol
 - ↓
 - Cipher Suite
 - ↓
 - Key Exchange

This provides runtime evidence.

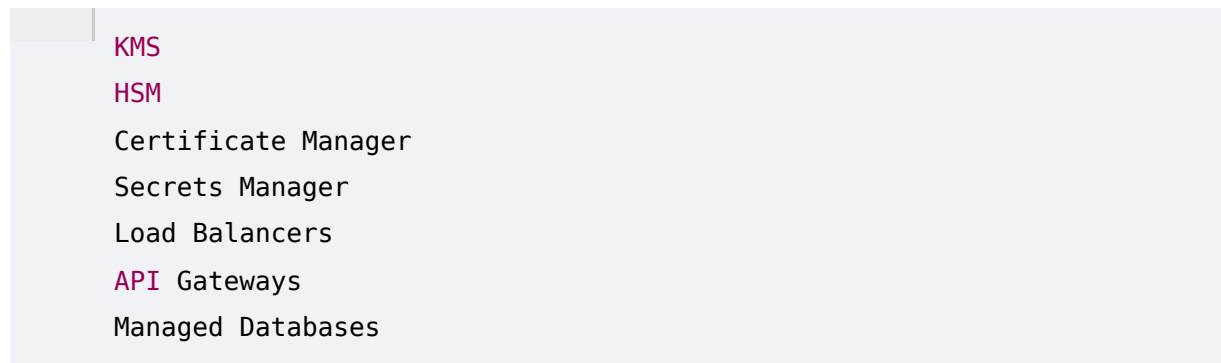
Important:

- Network scanning must be authorized and carefully controlled.

16. Cloud Scanner

Cloud environments introduce additional cryptographic assets.

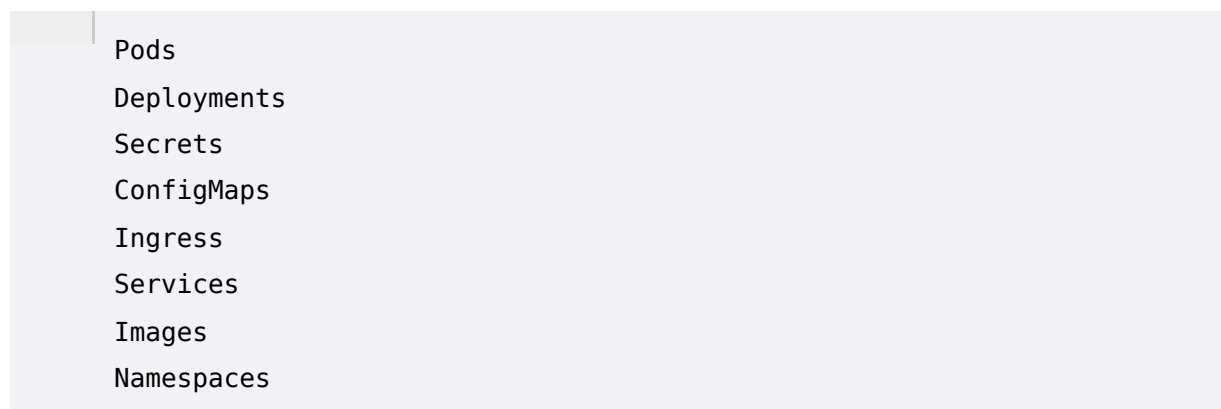
Examples:



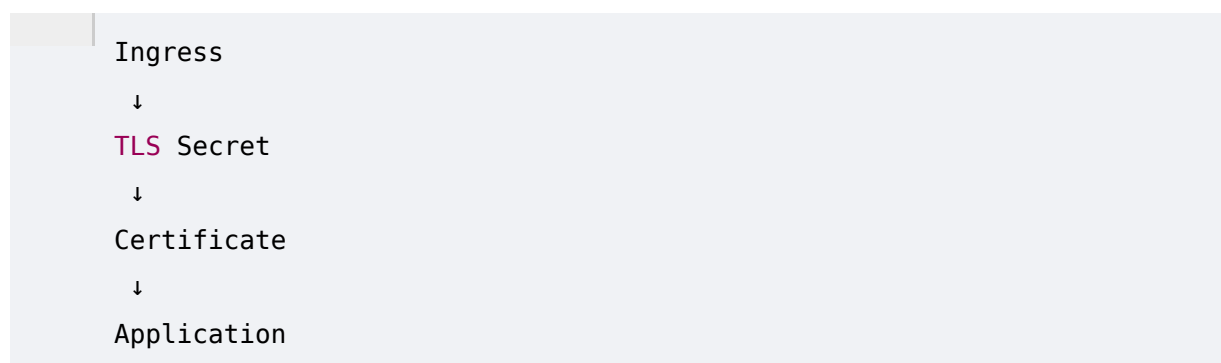
ECDAT can use cloud APIs to discover metadata.

17. Kubernetes Scanner

Kubernetes contains:



ECDAT can correlate:



This is highly valuable.

18. HSM/KMS Scanner

Enterprise key-management infrastructure should be represented.

Metadata may include:

- Key ID
- Key Type
- Algorithm
- Key State
- Purpose
- Rotation
- Location
- Provider

The scanner must never retrieve private key material merely for inventory.

19. Runtime Scanner

Static analysis tells us:

- What could happen?

Runtime telemetry can tell us:

- What actually happened?

Potential evidence:

- Crypto API invocation
- TLS handshake
- Certificate usage
- Library loading
- Algorithm selection

Runtime instrumentation can greatly improve confidence.

20. SBOM Integration

Software Bills of Materials can provide:

- Component
- Version
- Dependency
- License
- Package

ECDAT can enrich SBOM information with:

```
Cryptographic Usage
Quantum Status
Risk
PQC Recommendation
```

This creates:

Cryptographic SBOM intelligence.

21. CI/CD Integration

ECDAT should eventually integrate into pipelines.

Example:

```
Developer
  ↓
Git Push
  ↓
CI/CD
  ↓
ECDAT Scan
  ↓
Crypto Findings
  ↓
Policy Check
  ↓
Build Decision
```

Potential policy:

```
Block new deployment if:
Critical quantum-vulnerable crypto
is introduced.
```

22. Endpoint Agent

For deeper enterprise visibility, ECDAT may eventually deploy agents.

The agent can report:

```
Libraries
Certificates
Processes
Crypto configuration
Runtime observations
```

However, agents increase:

- Deployment complexity
- Security risk
- Operational overhead

Therefore they may be a later-stage feature.

23. Passive Discovery

Passive discovery observes existing information without actively probing systems.

Examples:

```
Logs
Telemetry
Existing inventories
Network metadata
Certificates
SBOMs
CI/CD outputs
```

Advantages:

- Lower operational risk
- Less network impact

Disadvantage:

- May miss unknown systems.
-

24. Active Discovery

Active discovery interacts with systems.

Examples:

```
TLS handshake
Service interrogation
API discovery
```

Cloud **API** calls
Repository cloning

Advantages:

- More complete information

Risks:

- Operational impact
- Permission requirements
- Network restrictions

25. Offline Discovery

Some environments cannot connect to the central platform.

ECDAT should support:

```
Offline Scan
↓
Encrypted Result Package
↓
Transfer
↓
Central Ingestion
```

This is particularly important for restricted environments.

26. Air-Gapped Environments

For highly isolated systems:

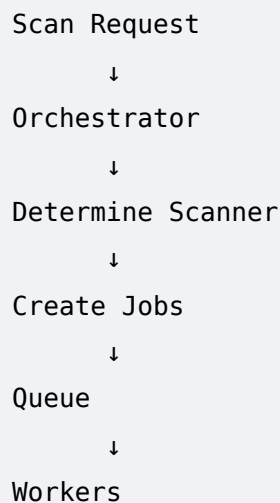
```
Air-Gapped Enterprise
|
▼
Local ECDAT Scanner
|
▼
Signed Result Bundle
|
▼
Controlled Transfer
|
```


The scanner should operate without internet access.

27. Scan Orchestration

With many scanners, ECDAT needs an orchestrator.

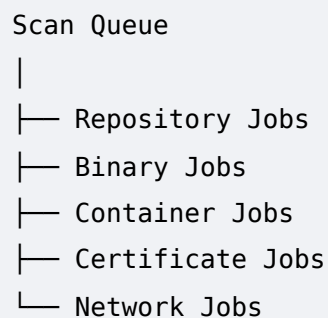
Conceptually:



28. Job Queue

A message queue can distribute work.

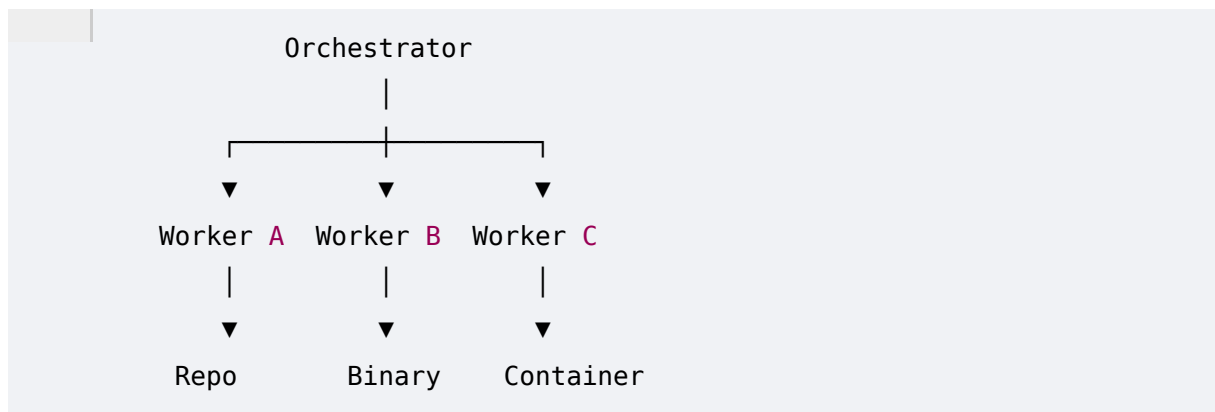
Example:



Workers consume jobs independently.

29. Distributed Workers

For enterprise scale:

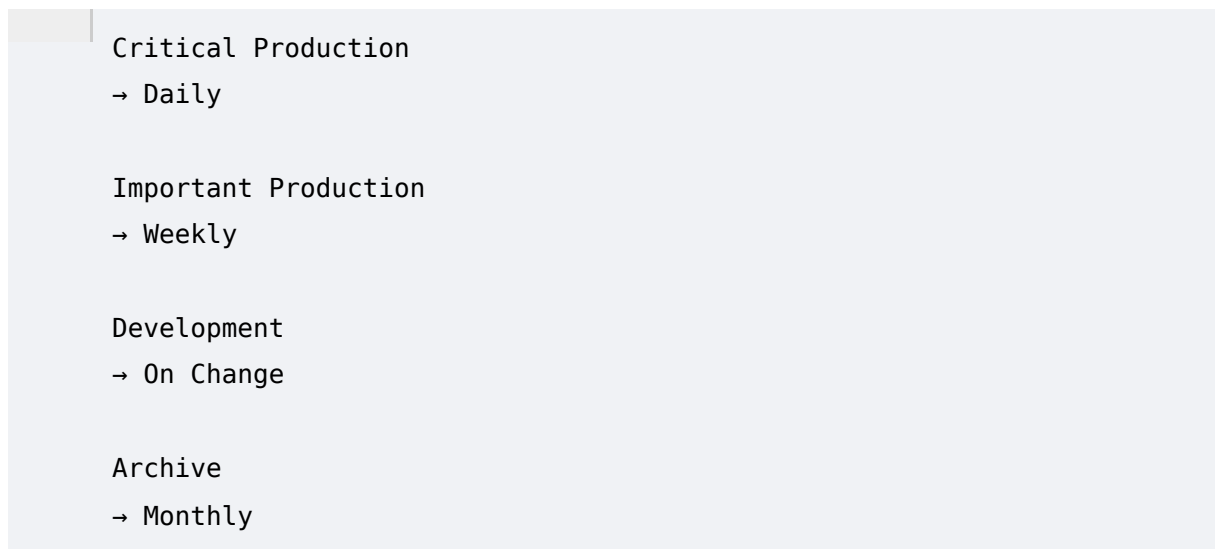


Workers can scale horizontally.

30. Scan Prioritization

Not every asset deserves equal scanning frequency.

For example:

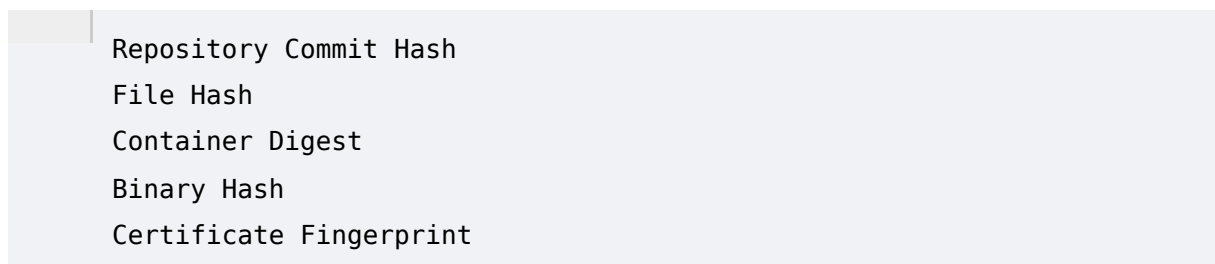


The exact cadence should be configurable.

31. Incremental Scanning

Avoid rescanning unchanged content.

Use:



If nothing changed:

```
Skip expensive analysis.
```

This saves huge amounts of compute.

32. Scheduled Scanning

ECDAT should support:

```
Hourly
Daily
Weekly
Monthly
Custom
```

Different scan types may use different schedules.

33. Event-Driven Scanning

Instead of waiting for a schedule:

```
Git Push
↓
Scan
```

or:

```
New Container Image
↓
Scan
```

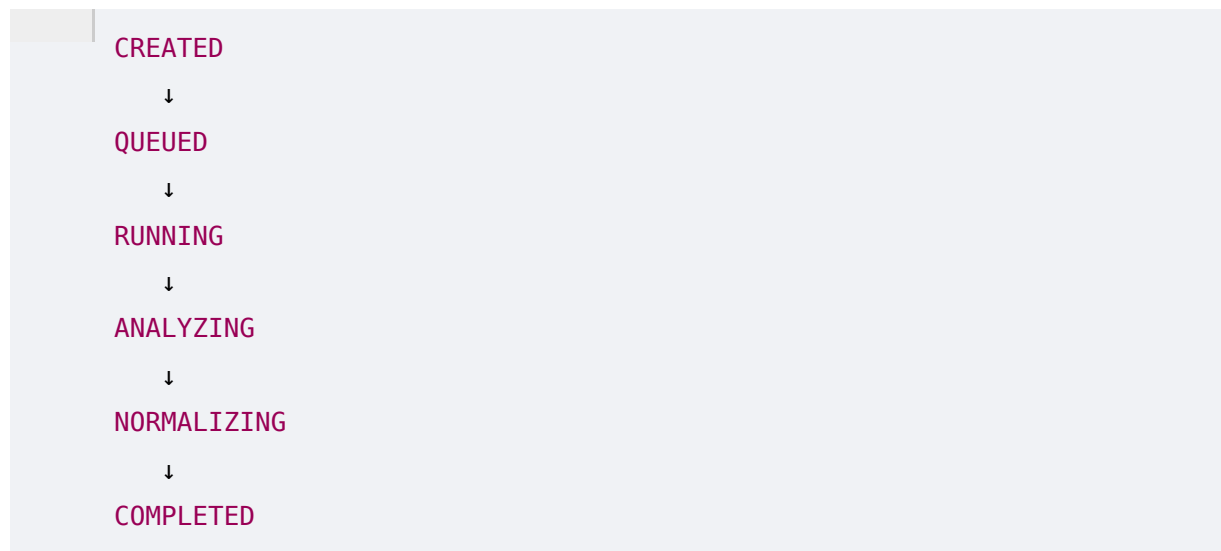
or:

```
Certificate Created
↓
Analyze
```

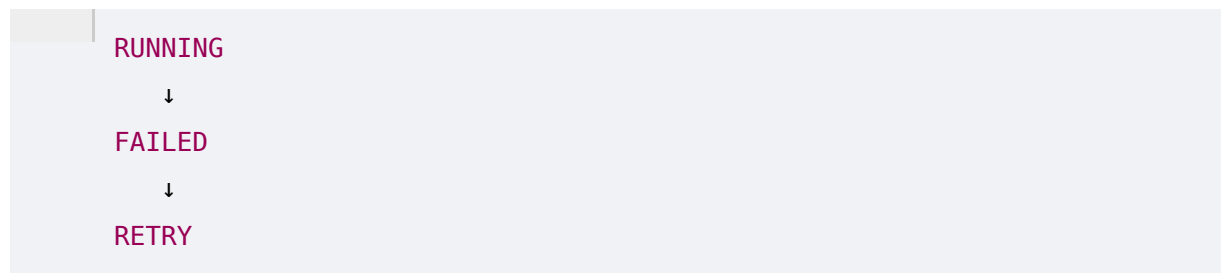
This allows near-real-time cryptographic inventory.

34. Scan Lifecycle

Every scan should have states:

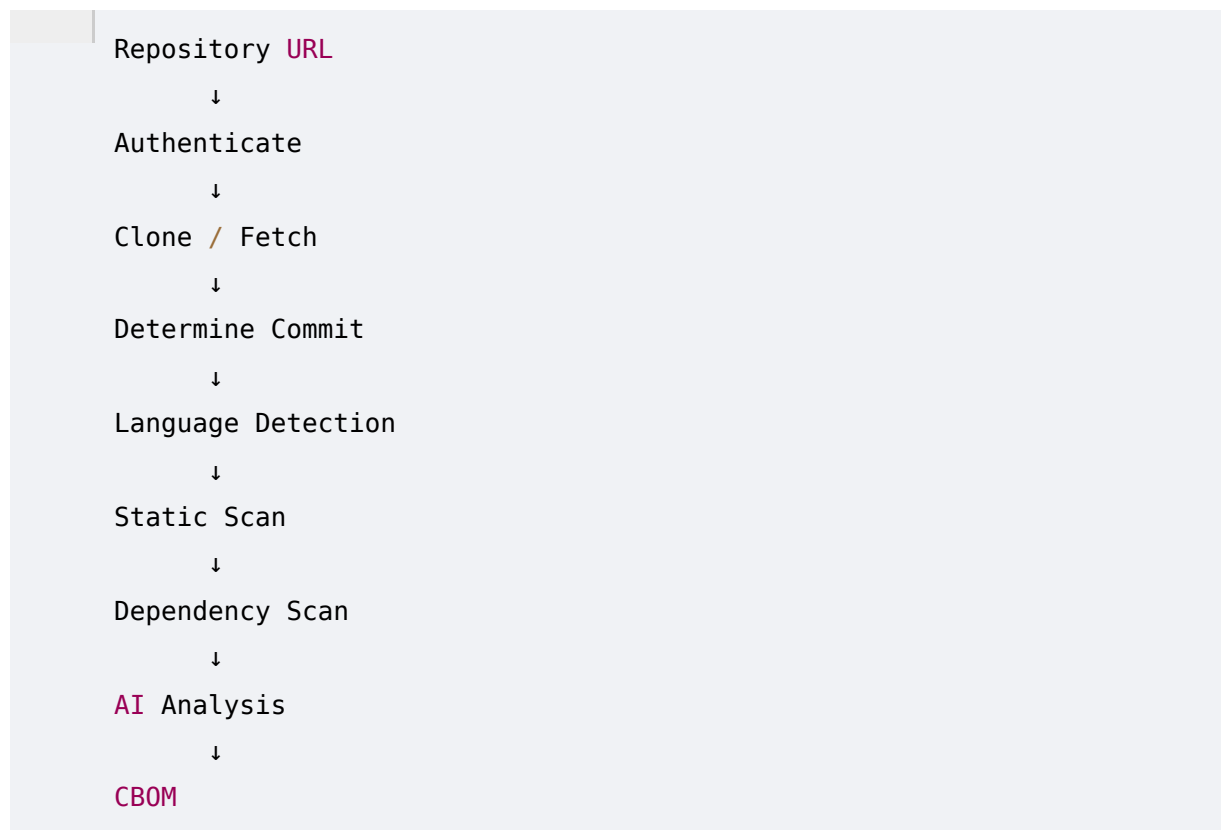


Failure:

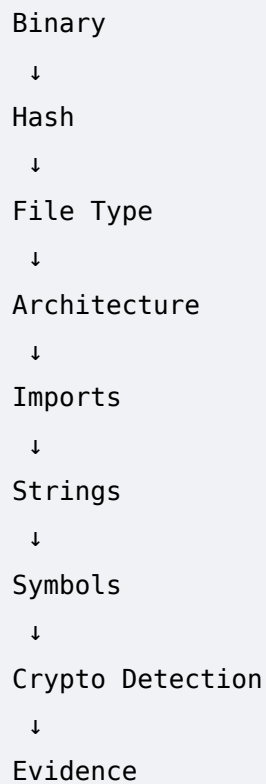


35. Repository Scan Lifecycle

Example:



36. Binary Scan Lifecycle



```
graph TD; A[Binary] --> B[Hash]; B --> C[File Type]; C --> D[Architecture]; D --> E[Imports]; E --> F[Strings]; F --> G[Symbols]; G --> H[Crypto Detection]; H --> I[Evidence];
```

Binary
↓
Hash
↓
File Type
↓
Architecture
↓
Imports
↓
Strings
↓
Symbols
↓
Crypto Detection
↓
Evidence

37. Container Scan Lifecycle



```
graph TD; A[Image] --> B[Digest]; B --> C[Manifest]; C --> D[Layers]; D --> E[Packages]; E --> F[Libraries]; F --> G[Application]; G --> H[Crypto Detection]; H --> I[CBOM];
```

Image
↓
Digest
↓
Manifest
↓
Layers
↓
Packages
↓
Libraries
↓
Application
↓
Crypto Detection
↓
CBOM

38. Network Scan Lifecycle

```
Target
↓
Authorization Check
↓
Service Discovery
↓
TLS Detection
↓
Certificate
↓
Protocol
↓
Crypto Parameters
↓
Evidence
```

Again, active scanning must only occur within authorized scope.

39. Evidence Collection

Every scanner should collect evidence.

Example:

```
Evidence Type:
Source Code

Location:
src/crypto.py:42

Observed:
AESGCM.encrypt()

Confidence:
0.99
```

Another:

```
Evidence Type:
Certificate
```

Fingerprint:

...

Observed:

ECDSA P-256

40. Evidence Normalization

Different scanners may report:

```
"RSA2048"
```

```
"RSA-2048"
```

```
"rsaEncryption / 2048"
```

The normalization layer converts these into:

```
Algorithm:
```

```
RSA
```

```
Parameter:
```

```
2048
```

This is essential for correlation.

41. Evidence Deduplication

Suppose:

```
Source scanner → RSA
```

```
Binary scanner → RSA
```

```
Runtime scanner → RSA
```

```
Certificate scanner → RSA
```

ECDAT should create:

```
ONE CRYPTO ASSET
```

with multiple evidence sources.

This increases confidence.

42. Asset Identity

Every asset needs a stable identifier.

Possible:

```
crypto_asset_id
certificate_id
application_id
library_id
```

The ID should remain stable across scans.

43. Asset Fingerprinting

Possible fingerprints:

Certificate

```
SHA-256 fingerprint
```

Container

```
Image digest
```

Source

```
Repository + path + commit
```

Library

```
Package + version
```

44. Version Tracking

ECDAT should detect:

```
OpenSSL 3.0
↓
OpenSSL 3.2
```

or:

```
RSA
↓
```


This allows migration tracking.

45. Change Detection

A new scan might discover:

New Crypto Finding

or:

Existing Crypto Removed

or:

Algorithm Changed

These events should trigger downstream analysis.

46. Scanner Confidence

Different scanners provide different confidence.

Example:

Certificate Parser:

0.99

Static API Detection:

0.95

Binary Heuristic:

0.71

AI Inference:

0.64

The evidence fusion layer can combine them.

47. Coverage Metrics

ECDAT should report:

```
Repositories Scanned
Containers Scanned
Binaries Scanned
Certificates Discovered
Cloud Resources Scanned
Endpoints Scanned
```

More importantly:

Discovery Coverage.

For example:

```
Estimated Enterprise Coverage:
84%
```

The methodology behind this estimate must be explicit.

48. Scanner Health

A scanner can fail silently.

ECDAT should monitor:

```
Last Successful Scan
Failure Rate
Average Duration
Queue Depth
Worker Health
Coverage
```

49. Failure Recovery

Suppose a repository scan fails.

The system should:

```
Detect Failure
↓
Record Error
↓
Retry
↓
Backoff
```

```
↓  
Retry Again  
↓  
Escalate
```

Do not silently mark failed scans as complete.

50. Retry Strategy

A reasonable approach:

```
Attempt 1  
↓  
Short Delay  
↓  
Attempt 2  
↓  
Longer Delay  
↓  
Attempt 3
```

Use exponential backoff where appropriate.

Certain errors should not be retried indefinitely:

```
Authentication Failure  
Invalid Repository  
Permission Denied
```

51. Credential Management

ECDAT needs secure credentials for:

- Git repositories
- Cloud APIs
- Kubernetes
- Network scanners
- HSMs
- Databases

Credentials should never be hardcoded into scanner configuration.

52. Secrets

Secrets should be stored using a secure secret-management mechanism.

Examples:

```
Vault
Cloud Secret Manager
KMS-backed secret store
```

The exact implementation depends on deployment environment.

53. Connector Security

Every connector should follow:

```
Least Privilege
+
Scoped Credentials
+
Short-lived Tokens
+
Audit Logs
```

Example:

A Git connector should have read-only access if scanning does not require writes.

54. Repository Authentication

Possible mechanisms:

```
SSH Deploy Key
OAuth
Personal Access Token
App Installation
```

ECDAT should prefer narrowly scoped credentials.

55. Cloud Authentication

Cloud scanners should use:

```
IAM Role
Service Account
Workload Identity
Temporary Credentials
```

rather than long-lived administrator credentials.

56. Network Permissions

Network scanning should be:

```
Explicitly Authorized
Scope-Limited
Rate-Limited
Audited
```

ECDAT should support target allowlists.

57. Least Privilege

The scanner should never request:

```
Full Administrator
```

when it only needs:

```
Read Certificates
List Keys
Read Metadata
```

This is especially important for an NTRO-oriented security tool.

58. Data Protection

Scan results may contain:

```
Source Code
Infrastructure Metadata
Certificates
Internal URLs
Security Architecture
```

Therefore:

Encryption At Rest

+

Encryption In Transit

+

Access Control

+

Audit

should be mandatory.

59. Air-Gapped Architecture

A complete air-gapped model:

ISOLATED NETWORK

Local Scanner

Local Evidence DB

Signed Export

Controlled Transfer



CENTRAL ECDAT

This makes the architecture suitable for highly restricted environments.

60. Large Enterprise Scale

Imagine:

10,000 repositories

100,000 containers

1 million certificates

millions of files

A single scanner process cannot handle this efficiently.

Therefore ECDAT should use:

Distributed Workers

+

Queues

+

Parallel Scanning

+

Caching

+

Incremental Analysis

61. Performance

Performance should be optimized at several layers.

I/O

Repository and file reading.

Parsing

AST construction.

Analysis

Static analysis.

AI

Model inference.

Storage

Evidence ingestion.

Graph

Relationship creation.

62. Parallelism

Independent scan jobs can run in parallel.

Example:

Repo A ↘

Repo B ↘

Repo C → Worker Pool

```
Repo D ─|
Repo E ─|
```

This enables horizontal scaling.

63. Caching

Useful cache keys:

```
File Hash
Container Digest
Commit SHA
Dependency Version
Certificate Fingerprint
```

If the input has not changed, reuse the analysis.

64. Storage

ECDAT may need several storage layers.

```
PostgreSQL
→ Metadata

Object Storage
→ Scan Artifacts

Graph Database
→ Relationships

Vector Store
→ Semantic Retrieval

Cache
→ Temporary Results
```

This separation is architecturally clean.

65. Message Bus

For large deployments:


```
Scanner
  ↓
Message Bus
  ↓
Normalizer
  ↓
AI
  ↓
Graph
```

Potential technologies could include:

- Kafka
- RabbitMQ
- NATS
- Cloud-native queues

The prototype can use a simpler queue.

66. API Architecture

Frontend:

```
Web UI
  ↓
API Gateway
  ↓
Backend Services
```

Possible services:

```
Scan Service
Asset Service
Risk Service
Recommendation Service
Graph Service
AI Service
Report Service
```

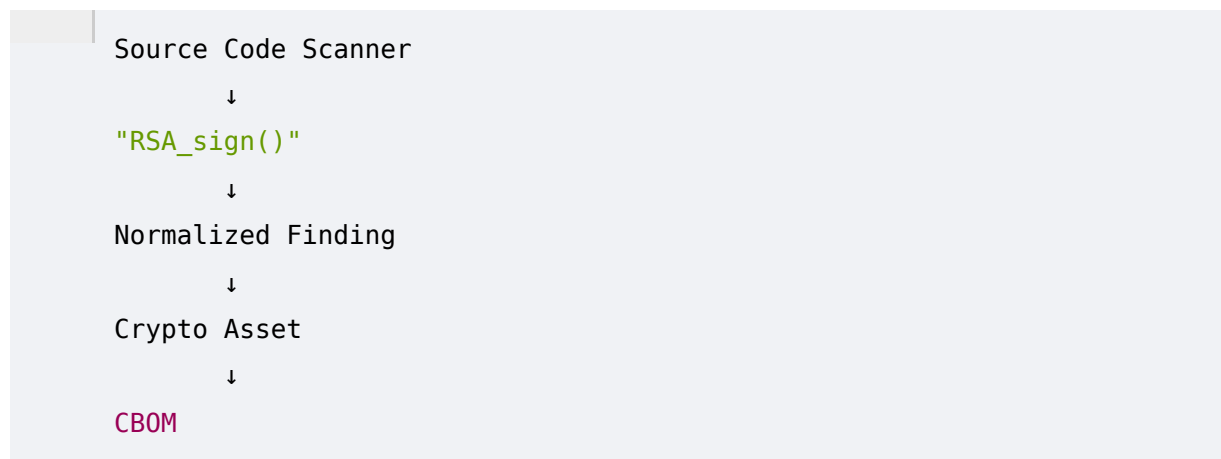
67. Scan Result Pipeline

The core pipeline:



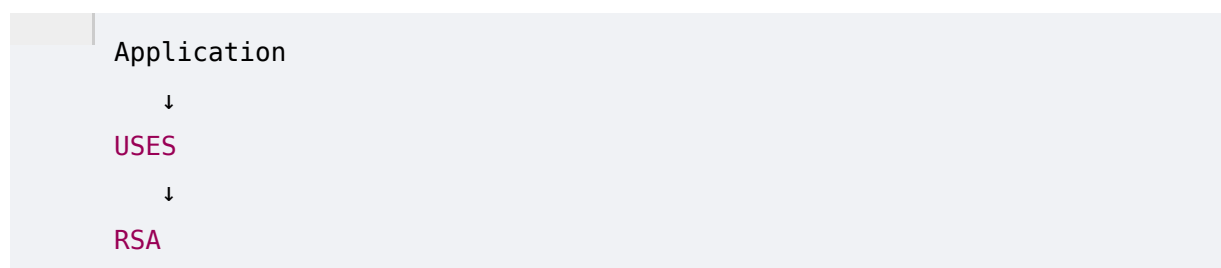
68. Scanner-to-CBOM Flow

Example:



69. Scanner-to-Graph Flow

The same finding creates relationships:



and:

```
Application
  ↓
DEPENDS_ON
  ↓
OpenSSL
```

70. Scanner-to-AI Flow

Only ambiguous findings should ideally reach expensive AI inference.

```
Static Scanner
  ↓
Ambiguous Candidate
  ↓
Context Builder
  ↓
LLM
  ↓
Validated Finding
```

This keeps the system efficient.

71. Scanner-to-Risk Flow

After normalization:

```
Crypto Asset
  ↓
Algorithm
  ↓
Quantum Status
  ↓
Data Context
  ↓
Risk Engine
```

72. Scan Prioritization Example

Suppose ECDAT has:

1,000 repositories

Prioritize:

Critical production repositories

↓

Security-sensitive repositories

↓

Public-facing services

↓

High-risk historical findings

↓

Everything else

This produces faster value.

73. Enterprise Example

Consider an enterprise:

5,000 Applications

↓

12,000 Repositories

↓

40,000 Containers

↓

200,000 Certificates

↓

15 HSMs

↓

3 Cloud Providers

ECDAT can create a unified inventory.

74. Sample Scan Result

Example:

SCAN RESULT

Repository:
payments-service

Commit:
a82f91d

Files:
1,248

Crypto Findings:
17

Confirmed:
13

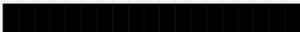
75. Coverage Dashboard

Example:

DISCOVERY COVERAGE

Repositories
 91%

Containers
 78%

Certificates
 96%

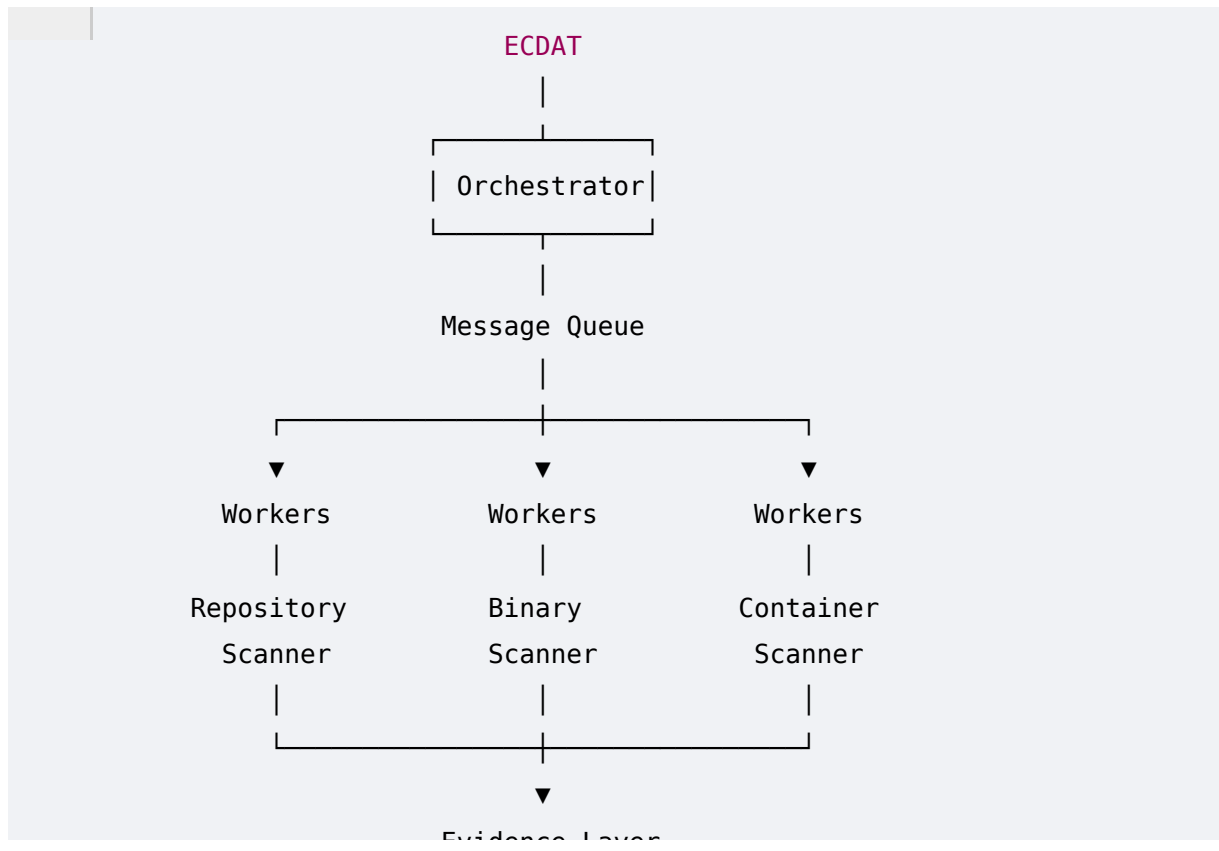
Cloud
 63%

Network
 51%

The dashboard should make coverage gaps visible.

76. Architecture

Complete enterprise scanning architecture:



77. Security Threat Model

The scanning infrastructure itself has threats.

Threat 1

Malicious repository.

Threat 2

Compromised scanner worker.

Threat 3

Credential theft.

Threat 4

Poisoned scan result.

Threat 5

Unauthorized network probing.

Threat 6

Sensitive source-code leakage.

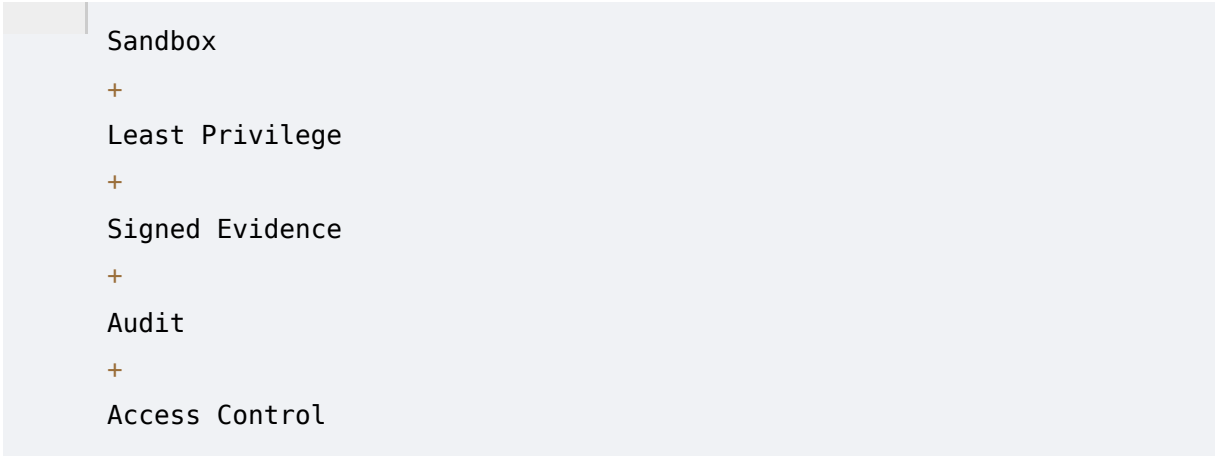
Threat 7

Graph poisoning.

Threat 8

AI prompt injection.

Therefore:



- + Sandbox
- + Least Privilege
- + Signed Evidence
- + Audit
- + Access Control

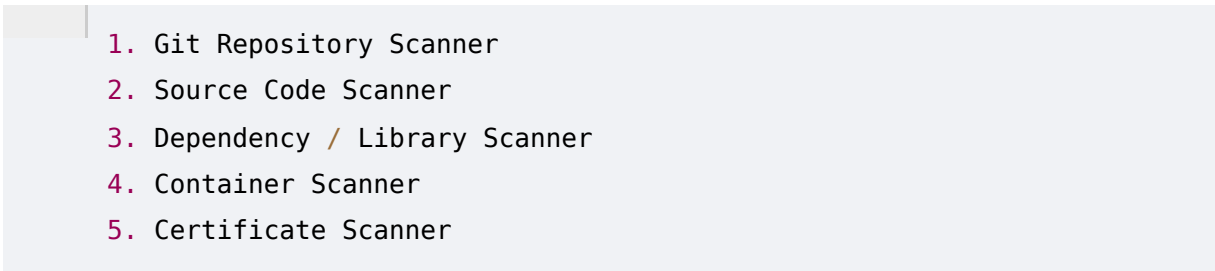
should be built into the architecture.

78. Requirements Derived From Phase 10

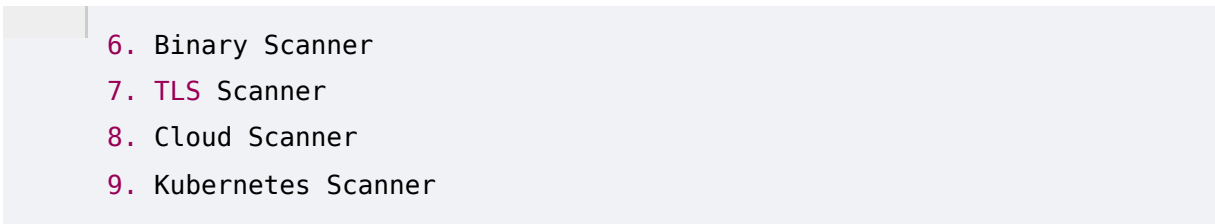
ECDAT should support a modular scanner framework.

Required MVP scanners

For the SIH prototype, prioritize:

- 
1. Git Repository Scanner
 2. Source Code Scanner
 3. Dependency / Library Scanner
 4. Container Scanner
 5. Certificate Scanner

Then add:

- 
6. Binary Scanner
 7. TLS Scanner
 8. Cloud Scanner
 9. Kubernetes Scanner

Advanced:

- 10. HSM/KMS
- 11. Runtime
- 12. Endpoint Agent

79. Phase 10 Summary

The key insight is:

A comprehensive CBOM cannot be created by a single scanner.

ECDAT needs a discovery fabric:

Repositories

+

Source

+

Binaries

+

Libraries

+

Containers

+

Certificates

+

Network

+

Cloud

+

Kubernetes

All evidence must converge into:

Normalized Evidence

↓

CBOM

↓

Knowledge Graph

↓

Risk

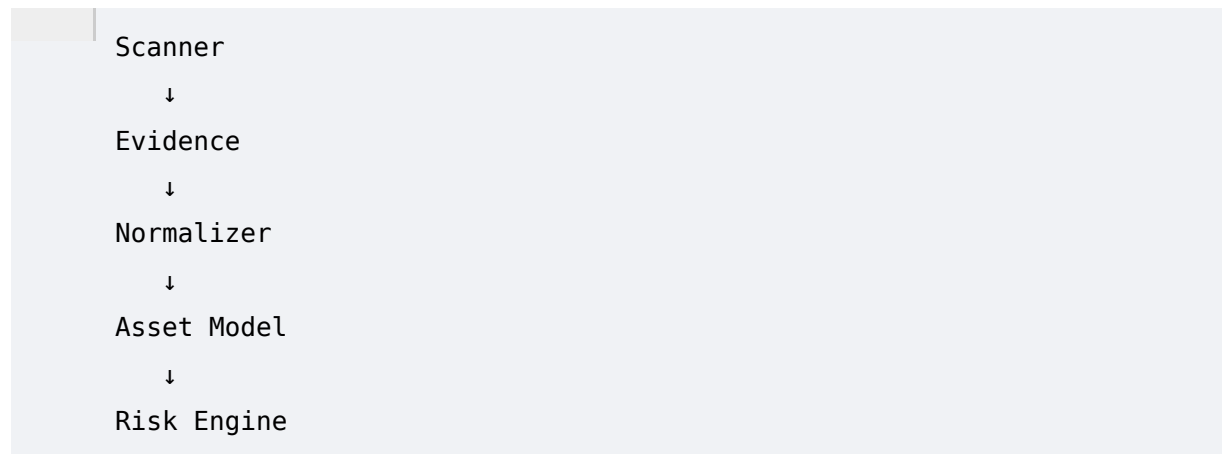
↓

Recommendation

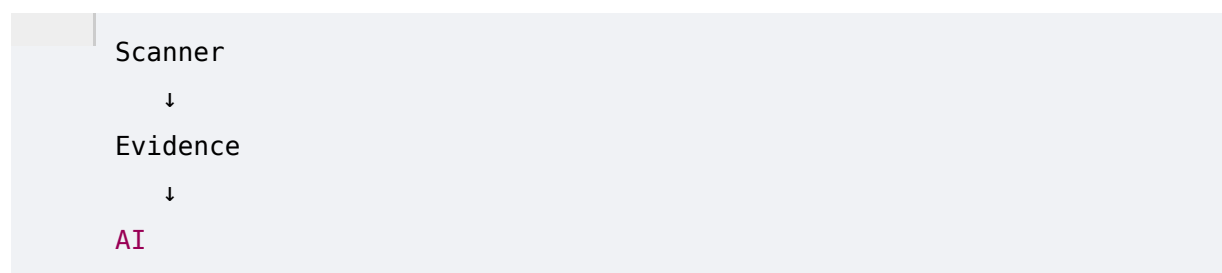
80. The Most Important Architectural Principle

The scanner should **not** directly decide risk.

Instead:

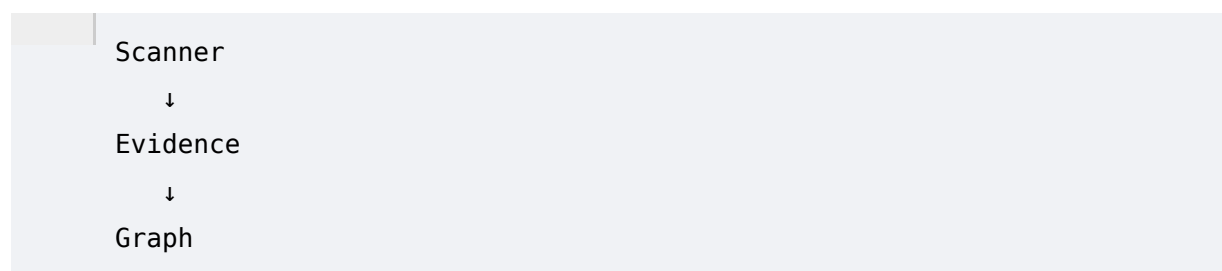


Likewise:



AI should not bypass the evidence model.

And:



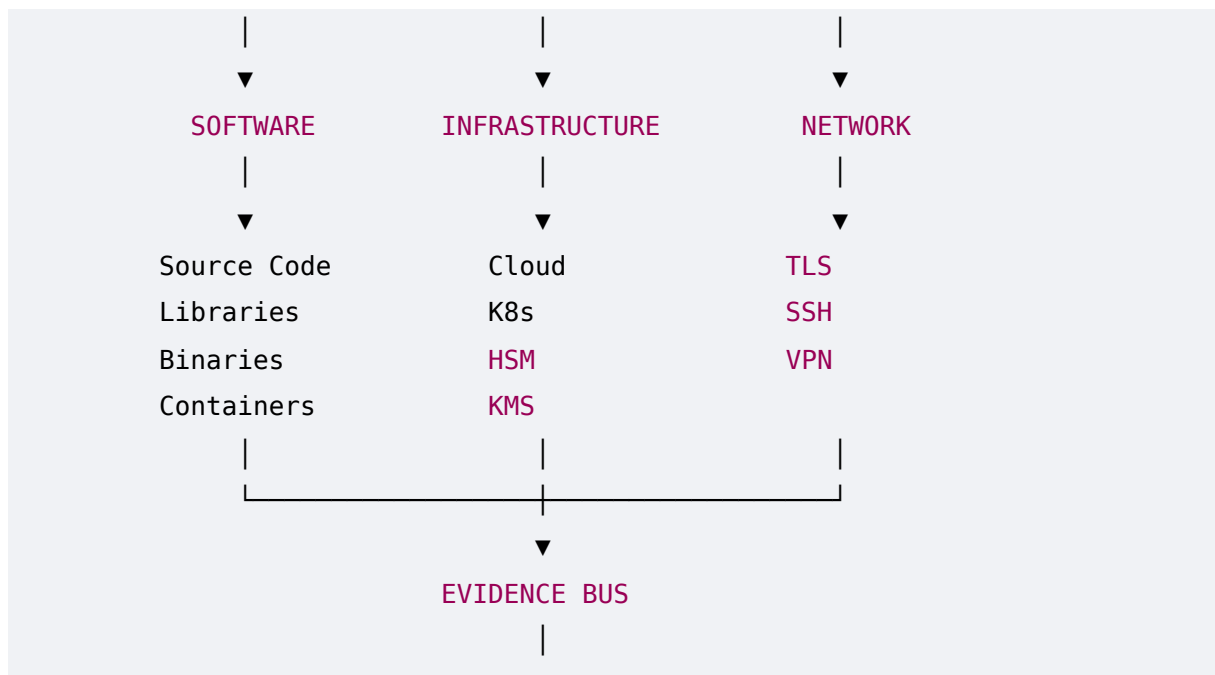
The graph should represent what is known and how confidently it is known.

This separation makes the system more defensible.

81. Final ECDAT Discovery Model

The discovery subsystem should ultimately look like:





This is the foundation on which the rest of ECDAT can be built.

PHASE 11 PREVIEW — CBOM DATA MODEL, SCHEMA & ASSET NORMALIZATION

We now have scanners producing evidence and a knowledge graph waiting to receive it.

The next major problem is:

What exactly does ECDAT store?

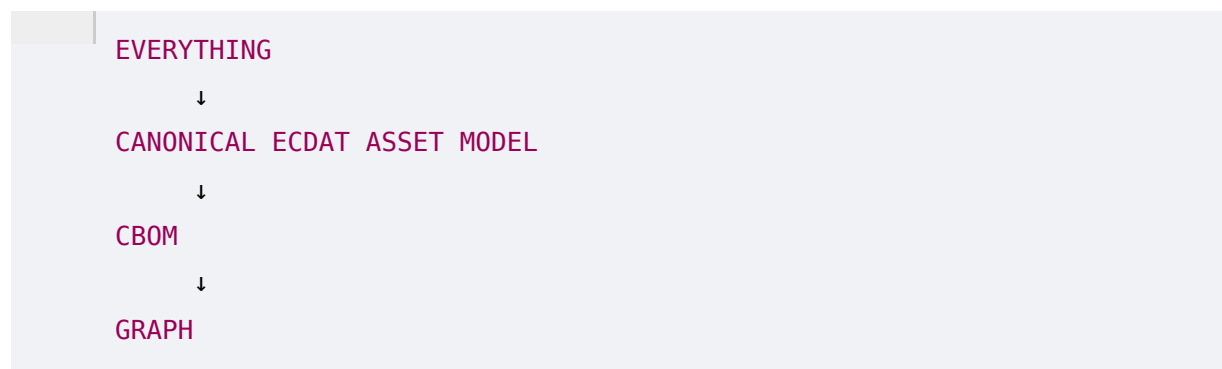
Phase 11 will design the actual data model:

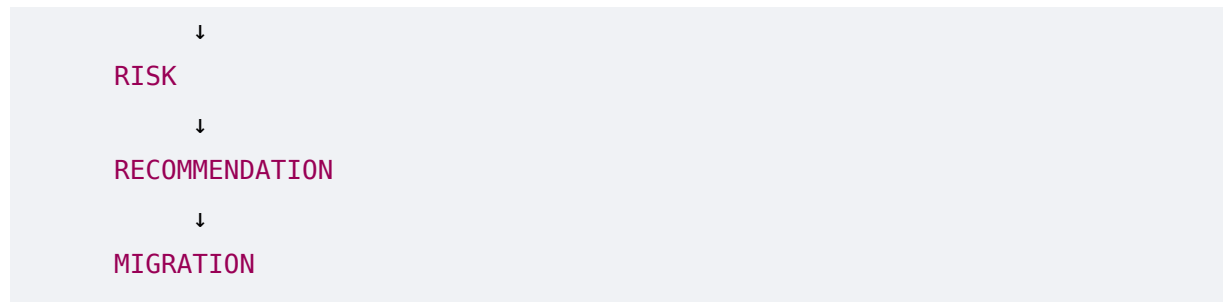
- What is a cryptographic asset?
- CBOM schema
- Asset identity
- Algorithm schema
- Key schema
- Certificate schema
- Library schema
- Protocol schema
- Application schema
- Container schema
- Evidence schema

- Finding schema
- Risk schema
- Recommendation schema
- Migration schema
- Relationship schema
- Versioning
- Provenance
- Confidence
- Normalization
- Deduplication
- CBOM interoperability
- CycloneDX considerations
- SPDX considerations
- Custom ECDAT extensions
- JSON representation
- Database schema
- Graph schema
- API objects
- Schema evolution
- Backward compatibility
- Data validation
- Import/export
- Enterprise reporting

The goal will be to define the **single canonical cryptographic asset model** that every scanner, AI component, risk engine, graph service, dashboard, and report uses.

In other words:





That schema becomes the **backbone of the entire platform**.

Phase 10 complete.